

2.04-hpi-nomogram-testing

September 2, 2024

```
[ ]: %load_ext autoreload
    %autoreload 2

import numpy as np
import pandas as pd
import torch
import pickle
from prism.config import PROCESSED_DATA_DIR, MODELS_DIR
from prism.legacy import normalise
from prism.load_models import load_mlp, load_prn
from prism.nomogram import nomogram
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

```
[43]: %reload_ext autoreload
```

```
[14]: seed = 257
      np.random.seed(seed)
      torch.manual_seed(seed)
```

```
[14]: <torch._C.Generator at 0x25ff86fa8f0>
```

0.1 Load and preprocess data

```
[15]: data_train = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_train.
      ↪CSV'))
      data_test = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_test.
      ↪CSV'))
      data_val = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_val.csv'))

      data_train_test = pd.concat([data_train,data_test]) # used for axis annotation,
      ↪in some plotting functions

      # drop id column
      data_train.drop('trr_id_code',axis=1,inplace=True)
      data_test.drop('trr_id_code',axis=1,inplace=True)
      data_val.drop('trr_id_code',axis=1,inplace=True)
```

```

target_col = 'oneyearmort'

x_train0 = data_train.drop(target_col,axis=1)
y_train = data_train[target_col]

x_test0 = data_test.drop(target_col,axis=1)
y_test = data_test[target_col]

x_val0 = data_val.drop(target_col,axis=1)
y_val = data_val[target_col]

[x_train,x_test] = normalise(x_train0,x_test0)
x_val = normalise(x_val0)

x_train_tensor = torch.tensor(x_train.values, dtype=torch.float32)
x_test_tensor = torch.tensor(x_test.values, dtype=torch.float32)

```

0.2 MLP nomogram

```

[16]: filename_mlp = 'mlp_model_20240705_135534'
try:
    mlp, mlp_params, mlp_metrics = load_mlp(filename_mlp, MODELS_DIR)
    print("Model loaded successfully")
    print("Derived Model Structure:")
    print("\nModel Parameters:")
    print(mlp_params)
    print("\nModel Metrics:")
    print(mlp_metrics)
except Exception as e:
    print(f"Error loading model: {e}")
    raise

# Extract method and device from mlp_params, with defaults
method = mlp_params.get('method', 'dirac')
device = mlp_params.get('device', 'cpu')

```

Model loaded successfully

Derived Model Structure:

Input dimension: 11

Hidden units: 10

Output dimension: 1

Model Parameters:

```
{'n_hidden': 10, 'weight_decay': 1e-05, 'lr': 0.001, 'patience': 50,
'tolerance': 0.0001, 'batch_size': 1024, 'device': 'cpu', 'seed': 257,
'input_dim': 11, 'hidden_units': 10, 'output_dim': 1}
```

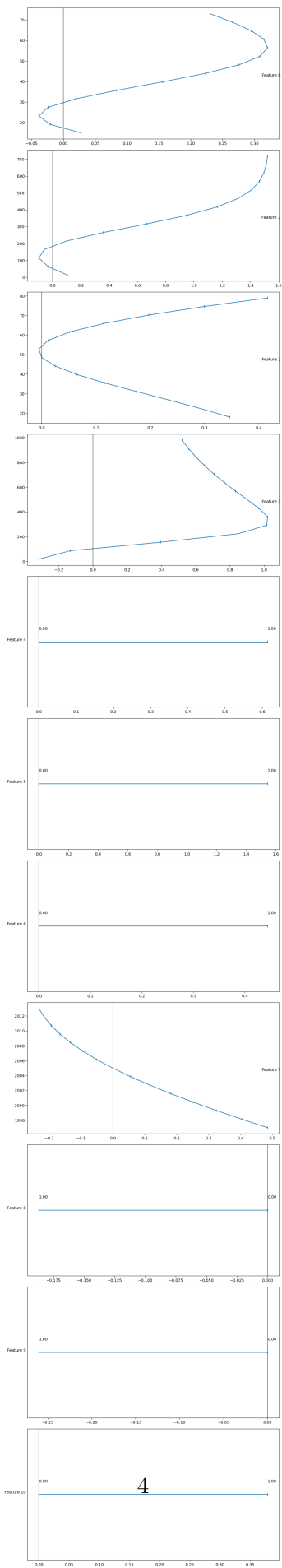
Model Metrics:

```
{'prevalence': 0.123, 'sensitivity': 0.241, 'specificity': 0.864, 'accuracy':  
0.797, 'ppv': 0.175, 'auc score': 0.601, 'auc lower ci': '0.577', 'auc upper  
ci': '0.626'}
```

```
[26]: x_train0_median = torch.tensor(x_train0.median().values, dtype=torch.float32,␣  
    ↪device=device)  
x_train0_std = torch.tensor(x_train0.std().values, dtype=torch.  
    ↪float32,device=device)
```

```
[18]: device='cpu'  
method='dirac'  
betas_univariate = torch.zeros([66,1], dtype=torch.float32,device=device)  
betas_univariate[:x_train.shape[1],0] = 1
```

```
[27]: univariate_responses, bivariate_responses, x_univariate, x_bivariate = nomogram(  
    betas_univariate, 0, x_train_tensor, x_train0_median, x_train0_std, mlp,  
    n_steps=15, sd_scale=2, method="dirac", device="cpu",␣  
    ↪categorical_threshold=15  
)
```



0.3 PRN nomogram

```
[44]: filename_prn = 'prn_model_20240705_140928'
try:
    prn, prn_params, prn_metrics = load_prn(filename_prn, MODELS_DIR)
    print("Model architecture:")
    print(prn)

    print("\nModel parameters:")
    for key, value in prn_params.items():
        print(f"{key}: {value}")

    print("\nModel metrics:")
    for key, value in prn_metrics.items():
        print(f"{key}: {value}")

except Exception as e:
    print(f"Error loading model: {e}")
    raise

# Extract method and device from prn_params, with defaults
method = prn_params.get('method', 'dirac')
device = prn_params.get('device', 'cpu')
```

Model architecture:

```
MaskedMLP(
  (fc1): Linear(in_features=11, out_features=55, bias=True)
  (activation): Tanh()
  (fc2): Linear(in_features=55, out_features=1, bias=True)
)
```

Model parameters:

```
n_hidden: 11
mask: [[1. 1. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 1. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 1. 1. 1. 1. 1.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 1. 1. 1. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0. 0.]
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 1. 1.
 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 1. 1. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 1. 1. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 1. 1. 1. 0. 0. 0.
 0. 0. 1. 1. 1. 1. 1.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 1.
 1. 1. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0.]
```

```
subnet_nodes: 5
iter: 10000
lr: 0.05
weight_decay: 1e-05
tolerance: 0.0001
patience: 100
device: cpu
seed: 257
```

```
Model metrics:
prevalence: 0.123
sensitivity: 0.235
specificity: 0.867
accuracy: 0.799
ppv: 0.174
auc score: 0.602
auc lower ci: 0.578
auc upper ci: 0.63
```

```
[ ]: univariate_responses_senn, bivariate_responses_senn, x_univariate_senn,
      ↪x_bivariate_senn = nomogram(
          betas_univariate, 0, x_train_tensor, x_train0_median, x_train0_std, prn,
          n_steps=15, sd_scale=2, method=method, device=device,
          ↪categorical_threshold=15
      )
```

```
[50]: n_univ = x_train.shape[1]
      n_biv = n_univ * (n_univ - 1) // 2
      n_features = n_univ + n_biv
      betas_all = torch.ones([n_features,1], dtype=torch.float32,device=device)
```

```
[ ]: univariate_responses_senn, bivariate_responses_senn, x_univariate_senn,
      ↪x_bivariate_senn = nomogram(
          betas_all, 0, x_train_tensor, x_train0_median, x_train0_std, prn,
          n_steps=15, sd_scale=2, method=method, device=device,
          ↪categorical_threshold=15
      )
```